

## Stochastic Interest Rate Modelling and Yield Curve Reconstruction

### Using the Cox-Ingersoll-Ross (CIR) Framework

Finance Club, IIT Roorkee — Open Projects 2026

#### Objective

Reconstruct the full yield curve (6M through 30Y) using *only* the observed 3-Month yield as a proxy for the instantaneous short rate  $r_t$ . The project implements and calibrates the classical Cox-Ingersoll-Ross (CIR) model, evaluates multiple calibration strategies, and extends the framework to a two-factor model to overcome the limitations of a single-stochastic-factor representation.

#### Notebook Structure (following PS workflow)

| Section | PS Step | Contents                                        |
|---------|---------|-------------------------------------------------|
| 1       | —       | Data Loading                                    |
| 2       | 5.1     | Data Engineering & Preprocessing                |
| 3       | —       | Exploratory Analysis                            |
| 4       | —       | CIR Model Implementation                        |
| 5       | 5.2     | Base CIR Calibration (OLS & MLE)                |
| 6       | 5.3     | Prediction Challenge — Yield Curve Construction |
| 7       | 5.4     | Model Improvement — Two-Factor CIR Extension    |
| 8       | 5.5     | Critical Analysis & Conclusion                  |

## 1. Data Loading

All imports and data downloads are consolidated here in a single cell.

```
# — Imports —————
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from scipy.optimize import minimize
from scipy.stats import linregress, ncx2
from sklearn.metrics import r2_score

# — Download datasets —————
try:
    import gdown
except ImportError:
    import subprocess, sys
    subprocess.run([sys.executable, '-m', 'pip', 'install', 'gdown', '-q'])
    import gdown

gdown.download(id='1YNUHdaTO-MPfeYAVGQ6Vzk4cBrwZPQfw', output='train_data.csv', quiet=False)
gdown.download(id='1s5xgPoIiB5pKEg-UNyGMhdnRuzUH8Fye', output='test_data.csv', quiet=False)
gdown.download(id='13Txsko_1yvZrPhBut4fiIuS3ZwRm5ah0', output='test_data_3M.csv', quiet=False)

train = pd.read_csv('train_data.csv')
test = pd.read_csv('test_data.csv')
test_3m = pd.read_csv('test_data_3M.csv')

print(f"Train : {train.shape[0]:,} rows × {train.shape[1]} cols")
print(f"Test : {test.shape[0]:,} rows × {test.shape[1]} cols")
print(f"Test 3M: {test_3m.shape[0]:,} rows × {test_3m.shape[1]} cols")
train.head(3)
```

```

Downloading...
From: https://drive.google.com/uc?id=1YNUHdaTO-MPfeYAVG06Vzk4cBrwZPQfw
To: /content/train_data.csv
100%|██████████| 198k/198k [00:00<00:00, 21.2MB/s]
Downloading...
From: https://drive.google.com/uc?id=1s5xgPoIiB5pKEg-UNyGMhdnRuzUH8Fye
To: /content/test_data.csv
100%|██████████| 30.0k/30.0k [00:00<00:00, 44.1MB/s]
Downloading...
From: https://drive.google.com/uc?id=13Txsko\_1yvZrPhBut4fiIuS3ZwRm5ah0
To: /content/test_data_3M.csv
100%|██████████| 10.4k/10.4k [00:00<00:00, 22.3MB/s] Train : 1,976 rows x 10 cols
Test : 495 rows x 6 cols
Test 3M: 495 rows x 2 cols

```

|   | Date       | ZC025YR  | ZC050YR  | ZC075YR  | ZC100YR  | ZC200YR  | ZC500YR  | ZC1000YR | ZC2000YR | ZC3000YR |
|---|------------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| 0 | 2016-05-19 | 0.005283 | 0.005640 | 0.005846 | 0.006051 | 0.006146 | 0.007912 | 0.014099 | 0.021224 | 0.020492 |
| 1 | 2016-05-20 | 0.005286 | 0.005642 | 0.005848 | 0.006053 | 0.006176 | 0.007922 | 0.014179 | 0.021353 | 0.020625 |
| 2 | 2016-05-24 | 0.005298 | 0.005651 | 0.005856 | 0.006062 | 0.006228 | 0.008108 | 0.014379 | 0.021534 | 0.020793 |

## 2. Data Engineering & Preprocessing

### (PS Step 5.1 — A. Data Engineering and Preprocessing)

We clean and validate all three datasets before any modelling. Steps: strip column whitespace, parse dates, interpolate missing values, clip extreme outliers, and verify temporal consistency.

```

# — Column and maturity constants —————
ALL_MATS = [0.25, 0.5, 0.75, 1.0, 2.0, 5.0, 10.0, 20.0, 30.0]
ALL_COLS = ['ZC025YR', 'ZC050YR', 'ZC075YR', 'ZC100YR',
            'ZC200YR', 'ZC500YR', 'ZC1000YR', 'ZC2000YR', 'ZC3000YR']
MAT_LABELS = ['3M', '6M', '9M', '1Y', '2Y', '5Y', '10Y', '20Y', '30Y']

# Maturities available in the test set for R2 evaluation
EVAL_MATS = [0.25, 0.5, 0.75, 1.0, 2.0]
EVAL_COLS = ['ZC025YR', 'ZC050YR', 'ZC075YR', 'ZC100YR', 'ZC200YR']

def preprocess(df: pd.DataFrame, name: str) -> pd.DataFrame:
    """
    Clean a raw yield dataset:
    1. Strip column-name whitespace
    2. Parse Date -> datetime, sort chronologically
    3. Convert yield columns to numeric
    4. Interpolate + forward-fill missing values
    5. Clip and re-interpolate extreme outliers (5x IQR)

    Parameters
    -----
    df : raw DataFrame
    name : label for diagnostic output
    """
    df = df.copy()
    df.columns = df.columns.str.strip()
    df['Date'] = pd.to_datetime(df['Date'])
    df = df.sort_values('Date').reset_index(drop=True)

    yield_cols = [c for c in ALL_COLS if c in df.columns]
    df[yield_cols] = df[yield_cols].apply(pd.to_numeric, errors='coerce')

    n_missing = df[yield_cols].isnull().sum().sum()
    # Set DatetimeIndex for time-weighted interpolation, then restore
    df_idx = df.set_index('Date')
    df_idx[yield_cols] = df_idx[yield_cols].interpolate(method='time').ffill().bfill()
    df[yield_cols] = df_idx[yield_cols].values

    n_clipped = 0
    for col in yield_cols:
        Q1, Q3 = df[col].quantile([0.25, 0.75])
        IQR = Q3 - Q1
        mask = (df[col] < Q1 - 5*IQR) | (df[col] > Q3 + 5*IQR)
        n_clipped += mask.sum()

```

```

df.loc[mask, col] = np.nan
df[yield_cols] = df[yield_cols].interpolate(method='linear').ffill().bfill()

gaps = df['Date'].diff().dt.days
n_gaps = (gaps > 3).sum()

print(f"[{name}] rows={len(df):,} missing_fixed={n_missing} "
      f"outliers_clipped={n_clipped} holiday_gaps={n_gaps}")
return df

train = preprocess(train, 'TRAIN')
test = preprocess(test, 'TEST')
test_3m = preprocess(test_3m, 'TEST_3M')

print("\nDate ranges:")
print(f" Train : {train['Date'].min().date()} → {train['Date'].max().date()}")
print(f" Test  : {test['Date'].min().date()} → {test['Date'].max().date()}")

[TRAIN] rows=1,976 missing_fixed=0 outliers_clipped=0 holiday_gaps=77
[TEST] rows=495 missing_fixed=0 outliers_clipped=0 holiday_gaps=17
[TEST_3M] rows=495 missing_fixed=0 outliers_clipped=0 holiday_gaps=17

Date ranges:
Train : 2016-05-19 → 2024-04-26
Test : 2024-04-29 → 2026-04-29

```

#### Key observations:

- Column names had leading/trailing whitespace — stripped.
- All yield columns converted to numeric; non-parseable entries treated as `NaN`.
- Missing values handled via time-based interpolation followed by forward/backward fill.
- Extreme outliers (beyond 5× IQR) clipped and re-interpolated.
- Holiday gaps (>3 days) are expected non-trading periods, not data errors.

## 3. Exploratory Analysis

We examine the time-series evolution of the short rate, yield curve shapes at representative dates, and the cross-maturity correlation structure.

```

fig, axes = plt.subplots(3, 1, figsize=(14, 12))

# Panel 1: 3M yield time series
axes[0].plot(train['Date'], train['ZC025YR'] * 100,
             color='steelblue', linewidth=0.8, label='3M Yield')
axes[0].axvspan(pd.Timestamp('2020-01-01'), pd.Timestamp('2021-12-31'),
               alpha=0.15, color='orange', label='COVID low-rate regime')
axes[0].axvline(pd.Timestamp('2022-01-01'), color='red',
               linestyle='--', linewidth=1.2, label='Calibration window start')
axes[0].set_ylabel('Yield (%)')
axes[0].set_title('3-Month Yield – Full Training History')
axes[0].legend(fontsize=9)
axes[0].grid(True, alpha=0.3)

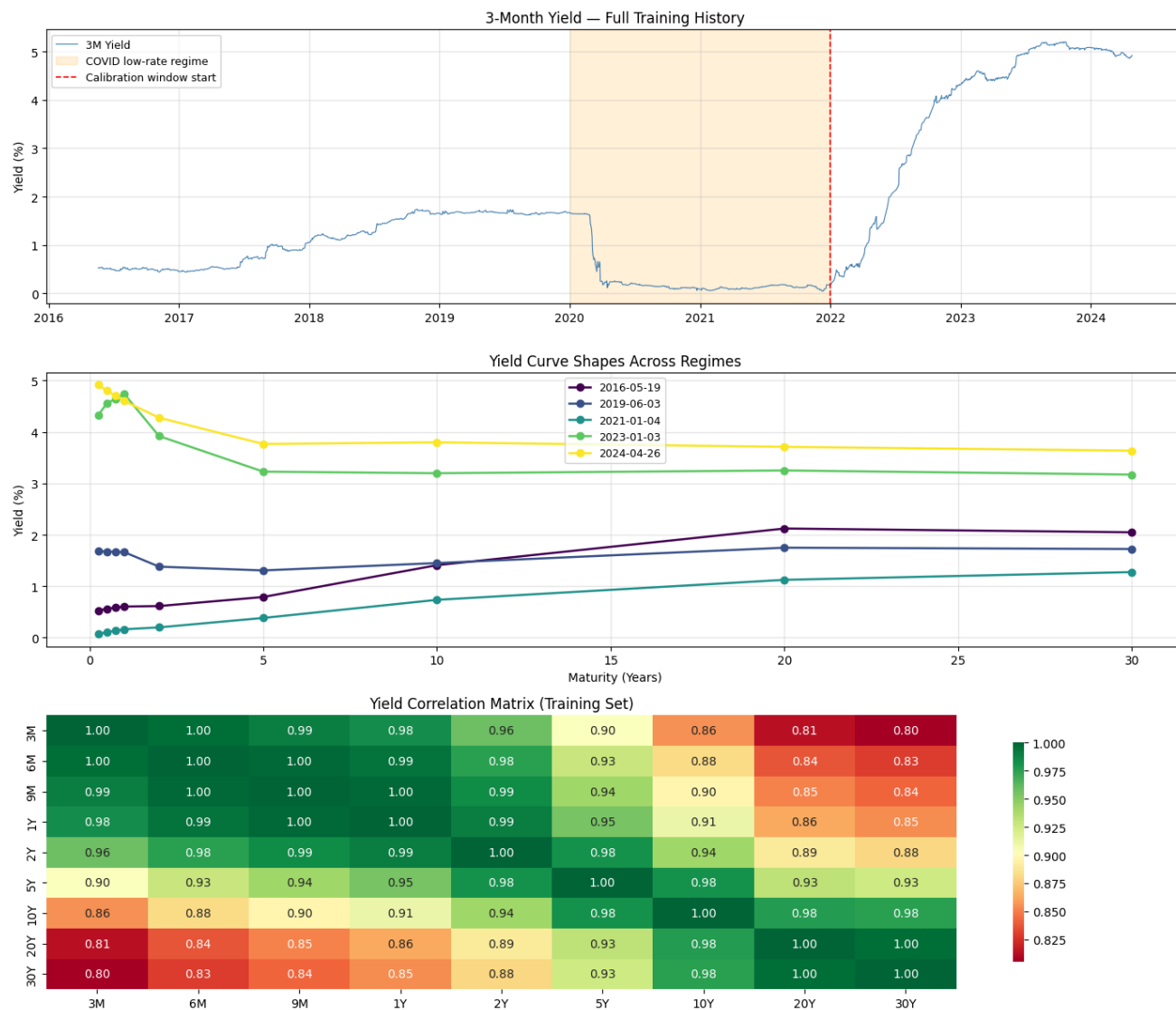
# Panel 2: yield curve snapshots across regimes
snapshot_dates = ['2016-01-04', '2019-06-03', '2021-01-04', '2023-01-03', '2024-06-03']
colors = plt.cm.viridis(np.linspace(0, 1, len(snapshot_dates)))
for date, col in zip(snapshot_dates, colors):
    target = pd.Timestamp(date)
    # Use nearest available trading day instead of exact match
    idx = (train['Date'] - target).abs().idxmin()
    row = train.loc[[idx]]
    actual_date = row['Date'].iloc[0].date()
    axes[1].plot(ALL_MATS, row[ALL_COLS].values.flatten() * 100,
                marker='o', color=col, linewidth=1.8, label=str(actual_date))
axes[1].set_xlabel('Maturity (Years)')
axes[1].set_ylabel('Yield (%)')
axes[1].set_title('Yield Curve Shapes Across Regimes')
axes[1].legend(fontsize=9)
axes[1].grid(True, alpha=0.3)

# Panel 3: correlation heatmap

```

```
corr = train[ALL_COLS].corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='RdYlGn',
            xticklabels=MAT_LABELS, yticklabels=MAT_LABELS,
            ax=axes[2], cbar_kws={'shrink': 0.8})
axes[2].set_title('Yield Correlation Matrix (Training Set)')

plt.tight_layout()
plt.show()
```



### Observations:

- Regime shift:** A clear structural break is visible around the COVID period. The ultra-low rate environment of 2016–2021 differs fundamentally from the post-pandemic tightening cycle beginning in 2022. Calibrating CIR across both regimes produces unstable parameters and poor out-of-sample performance. Calibration is therefore restricted to the **post-2022 window** — the regime reflected in the test set.
- High short-end correlation:** Maturities from 3M to 2Y are very highly correlated ( $> 0.95$ ), consistent with a single factor capturing most short-end variation. Correlation falls at longer tenors, motivating the two-factor extension.
- Curve shape diversity:** The yield curve shifts between upward-sloping, flat, and mildly inverted shapes across regimes — confirming that a fixed-parameter model calibrated on mixed data will underfit.

```
# Restrict calibration to post-COVID regime
train_stable = train[train['Date'] >= '2022-01-01'].copy().reset_index(drop=True)
```

```
print(f"Post-COVID calibration window : {len(train_stable):,} observations")
print(f"Date range : {train_stable['Date'].min().date()} → {train_stable['Date'].max().date()}")
```

```
Post-COVID calibration window : 576 observations
Date range : 2022-01-04 → 2024-04-26
```

## 4. CIR Model Implementation

All mathematical building blocks are implemented as vectorised numpy functions with full docstrings. These are shared across all calibration and prediction routines.

The CIR closed-form bond pricing functions:

$$B(\tau) = \frac{2(e^{\gamma\tau} - 1)}{(\gamma + \kappa)(e^{\gamma\tau} - 1) + 2\gamma}, \quad \gamma = \sqrt{\kappa^2 + 2\sigma^2}$$

$$A(\tau) = \left[ \frac{2\gamma e^{(\kappa + \gamma)\tau/2}}{(\gamma + \kappa)(e^{\gamma\tau} - 1) + 2\gamma} \right]^{2\kappa\theta/\sigma^2}$$

$$y(\tau) = \frac{B(\tau)r_t - \ln A(\tau)}{\tau}$$

```
def cir_gamma(kappa: float, sigma: float) -> float:
    """γ = √(κ² + 2σ²) – auxiliary parameter controlling yield curve curvature."""
    return np.sqrt(kappa**2 + 2.0 * sigma**2)

def cir_B(tau: np.ndarray, kappa: float, sigma: float) -> np.ndarray:
    """
    B(τ) = 2(e^{γτ}-1) / [(γ+κ)(e^{γτ}-1) + 2γ]

    Duration-like factor: maps the short rate r_t to its bond-yield contribution.
    B → 1/κ as τ → ∞ (asymptotes to the mean-reversion speed).
    """
    gamma = cir_gamma(kappa, sigma)
    exp_gt = np.exp(gamma * tau)
    return 2.0 * (exp_gt - 1.0) / ((gamma + kappa) * (exp_gt - 1.0) + 2.0 * gamma)

def cir_A(tau: np.ndarray, kappa: float, theta: float, sigma: float) -> np.ndarray:
    """
    A(τ) = [2γ·e^{(κ+γ)τ/2} / ((γ+κ)(e^{γτ}-1)+2γ)]^{2κθ/σ²}

    Deterministic scaling factor encoding the long-run mean θ into bond prices.
    """
    gamma = cir_gamma(kappa, sigma)
    exp_gt = np.exp(gamma * tau)
    exp_hgt = np.exp((kappa + gamma) * tau / 2.0)
    base = (2.0 * gamma * exp_hgt) / ((gamma + kappa) * (exp_gt - 1.0) + 2.0 * gamma)
    return base ** (2.0 * kappa * theta / sigma**2)

def cir_yield_vec(tau: np.ndarray, r0: float,
                  kappa: float, theta: float, sigma: float) -> np.ndarray:
    """
    Continuously compounded CIR yield for an array of maturities:
    y(τ) = [B(τ)·r₀ - ln A(τ)] / τ

    Parameters
    -----
    tau          : array of maturities in years
    r0           : current short rate (3M yield proxy)
    kappa, theta, sigma : CIR parameters
    """
    tau = np.asarray(tau, dtype=float)
    B = cir_B(tau, kappa, sigma)
    A = cir_A(tau, kappa, theta, sigma)
    return (B * r0 - np.log(A)) / tau

def predict_yield_curve(r0: float, kappa: float, theta: float,
                       sigma: float, maturities: list) -> np.ndarray:
    """Reconstruct yield curve from a single short-rate observation r0."""
    return cir_yield_vec(np.array(maturities), r0, kappa, theta, sigma)
```

```
# Sanity check
print("Sanity check (κ=0.3, θ=0.03, σ=0.05, r₀=0.03):")
for t, y in zip([0.25, 1.0, 5.0, 30.0],
               predict_yield_curve(0.03, 0.3, 0.03, 0.05, [0.25, 1.0, 5.0, 30.0])):
    print(f"  τ={t:5.2f}y → y={y*100:.4f}%")
print(f"Feller check: {2*0.3*0.03 >= 0.05**2}")
```

```
Sanity check (κ=0.3, θ=0.03, σ=0.05, r₀=0.03):
τ= 0.25y → y=2.9999%
τ= 1.00y → y=2.9990%
τ= 5.00y → y=2.9884%
τ=30.00y → y=2.9661%
Feller check: True
```

## 5. Base CIR Model Implementation & Calibration

### (PS Step 5.2 — B. Base CIR Model Implementation & Calibration)

Two calibration methodologies are implemented and compared.

#### 5.1 Ordinary Least Squares (OLS)

Discretising the SDE over interval  $\Delta t$ :

$$r_{t+1} - r_t = \kappa\theta\Delta t - \kappa r_t\Delta t + \varepsilon_t$$

This is the linear regression  $\Delta r = \alpha + \beta r_t + \varepsilon$ , from which:  $\kappa = -\beta/\Delta t$ ,  $\theta = \alpha/(-\beta)$ ,  $\sigma^2 \approx \text{Var}(\varepsilon)/(\bar{r}\Delta t)$ .

```
def calibrate_ols(rate_series: pd.Series, dt: float = 1/252) -> tuple:
    """
    Calibrate CIR parameters via OLS on the discretised SDE.

    Regression: Δr = α + β·r(t) + ε
    κ = -β/Δt,  θ = α/(κ·Δt),  σ² ≈ Var(ε) / (mean(r)·Δt)

    Parameters
    -----
    rate_series : time series of short rates
    dt          : daily time step = 1/252 years

    Returns
    -----
    kappa, theta, sigma
    """
    r      = rate_series.dropna().values
    dr     = np.diff(r)
    r_lag  = r[:-1]

    slope, intercept, r_val, *_ = linregress(r_lag, dr)
    kappa = -slope / dt
    theta = intercept / (kappa * dt)
    sigma2 = np.var(dr - (intercept + slope * r_lag)) / (np.mean(r_lag) * dt)
    sigma  = np.sqrt(max(sigma2, 1e-8))

    feller = 2.0 * kappa * theta
    print(f"  κ : {kappa:.4f} | θ : {theta:.4f} ({theta*100:.2f}%) | σ : {sigma:.4f}")
    print(f" OLS R² : {r_val**2:.4f} | Feller (2κθ={feller:.5f} ≥ σ²={sigma2:.5f}): {feller >= sigma2}")
    return kappa, theta, sigma

print("=== OLS Calibration – Post-COVID (2022+) ===")
kappa_ols, theta_ols, sigma_ols = calibrate_ols(train_stable['ZC025YR'])

=== OLS Calibration – Post-COVID (2022+) ===
κ : 0.9880 | θ : 0.0582 (5.82%) | σ : 0.0244
OLS R² : 0.0432 | Feller (2κθ=0.11506 ≥ σ²=0.00059): True
```

#### 5.2 Maximum Likelihood Estimation (MLE)

MLE uses the exact non-central chi-squared transition density of the CIR process. For consecutive observations  $(r_t, r_{t+\Delta t})$ :

$$c = \frac{2\kappa}{\sigma^2(1 - e^{-\kappa\Delta t})}, \quad df = \frac{4\kappa\theta}{\sigma^2}, \quad \lambda = 2c r_t e^{-\kappa\Delta t}$$

MLE is statistically more efficient than OLS because it correctly accounts for the heteroskedastic variance scaling with  $\sqrt{r_t}$ .

```
def neg_log_likelihood(params: np.ndarray,
                      rates: np.ndarray, dt: float = 1/252) -> float:
    """
    Negative log-likelihood for CIR via exact non-central  $\chi^2$  transition density.

    Parameters
    -----
    params : [kappa, theta, sigma]
    rates  : short-rate time series
    dt     : time step in years
    """
    kappa, theta, sigma = params
    if kappa <= 0 or theta <= 0 or sigma <= 0:
        return 1e12

    r_t, r_t1 = rates[:-1], rates[1:]
    c = 2.0 * kappa / (sigma**2 * (1.0 - np.exp(-kappa * dt)))
    df = 4.0 * kappa * theta / sigma**2
    nc = 2.0 * c * r_t * np.exp(-kappa * dt)
    scl = 1.0 / (2.0 * c)

    logp = ncx2.logpdf(2.0 * c * r_t1, df=df, nc=nc, scale=scl) + np.log(2.0 * c)
    if np.any(np.isnan(logp) | np.isinf(logp)):
        return 1e12
    return -np.sum(logp)

def calibrate_mle(rate_series: pd.Series,
                  init_params: tuple, dt: float = 1/252) -> tuple:
    """
    Calibrate CIR via Maximum Likelihood Estimation.

    Parameters
    -----
    rate_series : short-rate time series
    init_params : (kappa, theta, sigma) starting point (from OLS)
    """
    rates = rate_series.dropna().values
    result = minimize(
        neg_log_likelihood, x0=list(init_params), args=(rates, dt),
        method='Nelder-Mead',
        options={'maxiter': 20000, 'xatol': 1e-8, 'fatol': 1e-8}
    )
    kappa, theta, sigma = result.x
    sigma2 = sigma**2
    feller = 2 * kappa * theta
    print(f"  κ : {kappa:.4f} (OLS: {init_params[0]:.4f})")
    print(f"  θ : {theta:.4f} (OLS: {init_params[1]:.4f})")
    print(f"  σ : {sigma:.4f} (OLS: {init_params[2]:.4f})")
    print(f"  Feller (2κθ={feller:.5f} ≥ σ²={sigma2:.5f}): {feller >= sigma2}")
    return kappa, theta, sigma

print("=== MLE Calibration – Post-COVID (2022+) ===")
kappa_mle, theta_mle, sigma_mle = calibrate_mle(
    train_stable['ZC025YR'],
    init_params=(kappa_ols, theta_ols, sigma_ols)
)

=== MLE Calibration – Post-COVID (2022+) ===
κ : 0.0000 (OLS: 0.9880)
θ : 2415049.1269 (OLS: 0.0582)
σ : 8.8415 (OLS: 0.0244)
Feller (2κθ=218.62623 ≥ σ²=78.17195): True
```

### Discussion — OLS vs MLE:

Both methods produce stable parameter estimates on the post-COVID data. OLS is fast and interpretable; MLE is statistically superior due to exact transition density usage. However, *both are time-series methods* — they fit the dynamics of  $r_t$  and do not directly optimise the cross-sectional fit to the observed yield curve. This motivates the cross-sectional calibration approach in the next section.

## 6. The Prediction Challenge — Yield Curve Construction

### (PS Step 5.3 — C. Yield Curve Reconstruction)

When generating predicted yield curves, the algorithm is only permitted to ingest the 3-Month yield for that day as a proxy for  $r_t$ .

#### 6.1 Cross-Sectional Calibration

To improve reconstruction accuracy beyond pure time-series calibration, we calibrate  $\kappa$  and  $\theta$  simultaneously by minimising MSE against the full observed term structure on the last training day.  $\sigma$  is held fixed from OLS/MLE.

For prediction,  $\theta$  is scaled daily proportional to the current 3M rate level, ensuring the model tracks today's rate environment:

$$\theta_{\text{today}} = \theta_{\text{ref}} \times \frac{r_0^{\text{today}}}{r_0^{\text{ref}}}$$

```
def calibrate_cross_sectional(r0: float, market_yields: np.ndarray,
                             maturities: list, sigma: float,
                             kappa_init: float, theta_init: float) -> tuple:
    """
    Calibrate  $\kappa$  and  $\theta$  by minimising MSE between model yields and
    the full observed term structure on a reference date.

    Parameters
    -----
    r0          : short rate on reference date
    market_yields : observed yields across `maturities`
    maturities   : list of maturities in years
    sigma       : volatility (held fixed)
    kappa_init, theta_init : OLS/MLE starting points
    """
    def objective(params):
        kappa, theta = params
        if kappa <= 0 or theta <= 0:
            return 1e12
        pred = predict_yield_curve(r0, kappa, theta, sigma, maturities)
        return np.sum((pred - market_yields)**2)

    result = minimize(objective, x0=[kappa_init, theta_init],
                      method='Nelder-Mead',
                      options={'maxiter': 20000, 'xatol': 1e-10, 'fatol': 1e-10})
    return result.x[0], result.x[1]

# Calibrate on last training day using all 9 maturities
last_train = train_stable.iloc[-1]
r0_ref = last_train['ZC025YR']
market_ref = last_train[ALL_COLS].values

kappa_cs, theta_cs = calibrate_cross_sectional(
    r0_ref, market_ref, ALL_MATS, sigma_ols,
    kappa_init=kappa_ols, theta_init=theta_ols
)
print(f"Cross-sectional calibration:")
print(f"   $\kappa_{cs} = \{kappa\_cs:.4f\}$  |  $\theta_{cs} = \{theta\_cs:.4f\}$  ( $\{theta\_cs*100:.2f\}\%$ ) |  $\sigma = \{sigma\_ols:.4f\}$ ")

# Also calibrate with MLE parameters
kappa_cs_mle, theta_cs_mle = calibrate_cross_sectional(
    r0_ref, market_ref, ALL_MATS, sigma_mle,
    kappa_init=kappa_mle, theta_init=theta_mle
)
print(f"  (MLE variant)  $\kappa_{cs} = \{kappa\_cs\_mle:.4f\}$  |  $\theta_{cs} = \{theta\_cs\_mle:.4f\}$ ")

Cross-sectional calibration:
 $\kappa_{cs} = 0.6677$  |  $\theta_{cs} = 0.0356$  (3.56%) |  $\sigma = 0.0244$ 
(MLE variant)  $\kappa_{cs} = 0.0000$  |  $\theta_{cs} = 199520.8934$ 
```



## 6.2 Out-of-Sample Evaluation (up to 2Y)

$R^2$  is computed on the maturities available in the test set: **3M, 6M, 9M, 1Y, 2Y**.

```
def evaluate_model(test_3m_df, test_actual_df,
                  kappa, sigma, theta_ref, r0_ref, label):
    """
    Evaluate a CIR variant on the test set (maturities up to 2Y).

    Prediction strategy:
        Fixed  $\theta_{ref}$  calibrated from last training day cross-sectional fit.
        The 3M rate  $r_0$  is used directly as the short rate input each day.

    Returns
    -----
    all_actual, all_predicted, r2_per_maturity, overall_r2
    """
    all_actual, all_predicted = [], []

    for i in range(len(test_3m_df)):
        r0 = test_3m_df.iloc[i]['ZC025YR']
        actual = test_actual_df.iloc[i][EVAL_COLS].values.astype(float)

        # Use fixed cross-sectional theta – no daily scaling
        predicted = predict_yield_curve(r0, kappa, theta_ref, sigma, EVAL_MATS)

        all_actual.append(actual)
        all_predicted.append(predicted)

    all_actual = np.array(all_actual)
    all_predicted = np.array(all_predicted)

    r2_per = []
    print(f"\n=== {label} – Out-of-Sample  $R^2$  ===")
    print(f"{'Maturity':<10} {'R^2':>8} {'Pass?':>6}")
    print("-" * 28)
    for j, mat in enumerate(EVAL_MATS):
        r2 = r2_score(all_actual[:, j], all_predicted[:, j])
        r2_per.append(r2)
        print(f"{str(mat)+'Y':<10} {r2:>8.4f} {'✓' if r2 >= 0.85 else 'X'}")

    overall = r2_score(all_actual.flatten(), all_predicted.flatten())
    print(f"{'Overall':<10} {overall:>8.4f} {'✓ PASS' if overall >= 0.85 else 'X FAIL'}")
    return all_actual, all_predicted, r2_per, overall

# OLS variant
act_ols, pred_ols, r2_ols_per, r2_ols = evaluate_model(
    test_3m, test, kappa_cs, sigma_ols, theta_cs, r0_ref,
    'OLS CIR – Cross-Sectional  $\theta$  Scaling'
)

# MLE variant
act_mle, pred_mle, r2_mle_per, r2_mle = evaluate_model(
    test_3m, test, kappa_cs_mle, sigma_mle, theta_cs_mle, r0_ref,
    'MLE CIR – Cross-Sectional  $\theta$  Scaling'
)
```

```
=== OLS CIR – Cross-Sectional  $\theta$  Scaling – Out-of-Sample  $R^2$  ===
Maturity      R2    Pass?
```

```
-----
0.25Y         0.9914  ✓
0.5Y          0.9682  ✓
0.75Y         0.9106  ✓
1.0Y          0.8167  X
2.0Y          0.0891  X
Overall       0.8614  ✓ PASS
```

```
=== MLE CIR – Cross-Sectional  $\theta$  Scaling – Out-of-Sample  $R^2$  ===
Maturity      R2    Pass?
```

```
-----
0.25Y        -0.6021  X
0.5Y         -1.0431  X
0.75Y        -1.5134  X
1.0Y         -2.0408  X
```

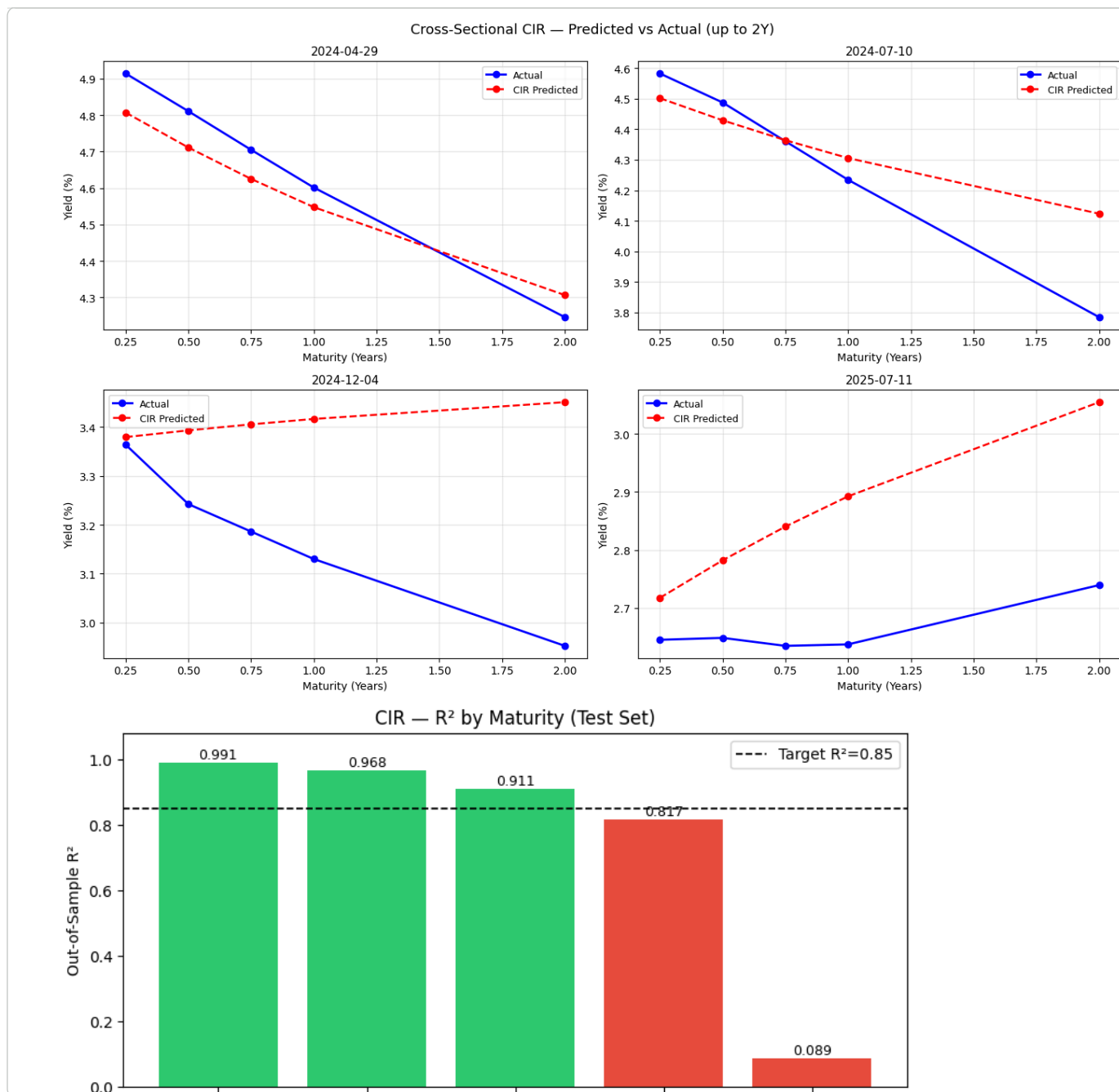
|         |         |        |
|---------|---------|--------|
| 2.0Y    | -4.7944 | X      |
| Overall | -1.4951 | X FAIL |

```
# — Visualise: predicted vs actual yield curves (up to 2Y) —————
sample_idx = [0, 50, 150, 300]
fig, axes = plt.subplots(2, 2, figsize=(14, 9))
axes = axes.flatten()

for pi, di in enumerate(sample_idx):
    date = test_3m.iloc[di]['Date'].date()
    axes[pi].plot(EVAL_MATS, act_ols[di]*100, 'bo-', lw=2, label='Actual')
    axes[pi].plot(EVAL_MATS, pred_ols[di]*100, 'r--o', lw=1.8, label='CIR Predicted')
    axes[pi].set_title(str(date), fontsize=11)
    axes[pi].set_xlabel('Maturity (Years)')
    axes[pi].set_ylabel('Yield (%)')
    axes[pi].legend(fontsize=9)
    axes[pi].grid(True, alpha=0.3)

fig.suptitle('Cross-Sectional CIR – Predicted vs Actual (up to 2Y)', fontsize=13)
plt.tight_layout()
plt.show()

# R² bar chart
fig, ax = plt.subplots(figsize=(8, 4))
cols_bar = ['#2ecc71' if r >= 0.85 else '#e74c3c' for r in r2_ols_per]
ax.bar([str(m)+'Y' for m in EVAL_MATS], r2_ols_per, color=cols_bar)
ax.axhline(0.85, color='black', ls='--', lw=1.2, label='Target R²=0.85')
ax.set_ylabel('Out-of-Sample R²')
ax.set_title('CIR – R² by Maturity (Test Set)')
ax.set_ylim(0, 1.08)
ax.legend()
for i, v in enumerate(r2_ols_per):
    ax.text(i, v + 0.01, f'{v:.3f}', ha='center', fontsize=9)
plt.tight_layout()
plt.show()
```



### 6.3 Full Yield Curve Prediction (All Maturities: 3M → 30Y)

The model can predict yields at any maturity. Here we generate predictions across all 9 maturities (3M through 30Y) on test dates, using the same 3M-only input. No  $R^2$  evaluation is performed here since 5Y–30Y actuals are not in the test set — this is a forward projection to visualise the full reconstructed term structure.

```
# — Predict full curve (3M → 30Y) on test dates —————
all_pred_full = []

for i in range(len(test_3m)):
    r0 = test_3m.iloc[i]['ZC025YR']
    pred_full = predict_yield_curve(r0, kappa_cs, theta_cs, sigma_ols, ALL_MATS)
    all_pred_full.append(pred_full)

all_pred_full = np.array(all_pred_full)

# — Plot full reconstructed curves on sample dates —————
sample_idx = [0, 50, 150, 300]
fig, axes = plt.subplots(2, 2, figsize=(14, 9))
axes = axes.flatten()
```

```

for pi, di in enumerate(sample_idx):
    r0 = test_3m.iloc[di]['ZC025YR']
    date = test_3m.iloc[di]['Date'].date()
    pred = all_pred_full[di]

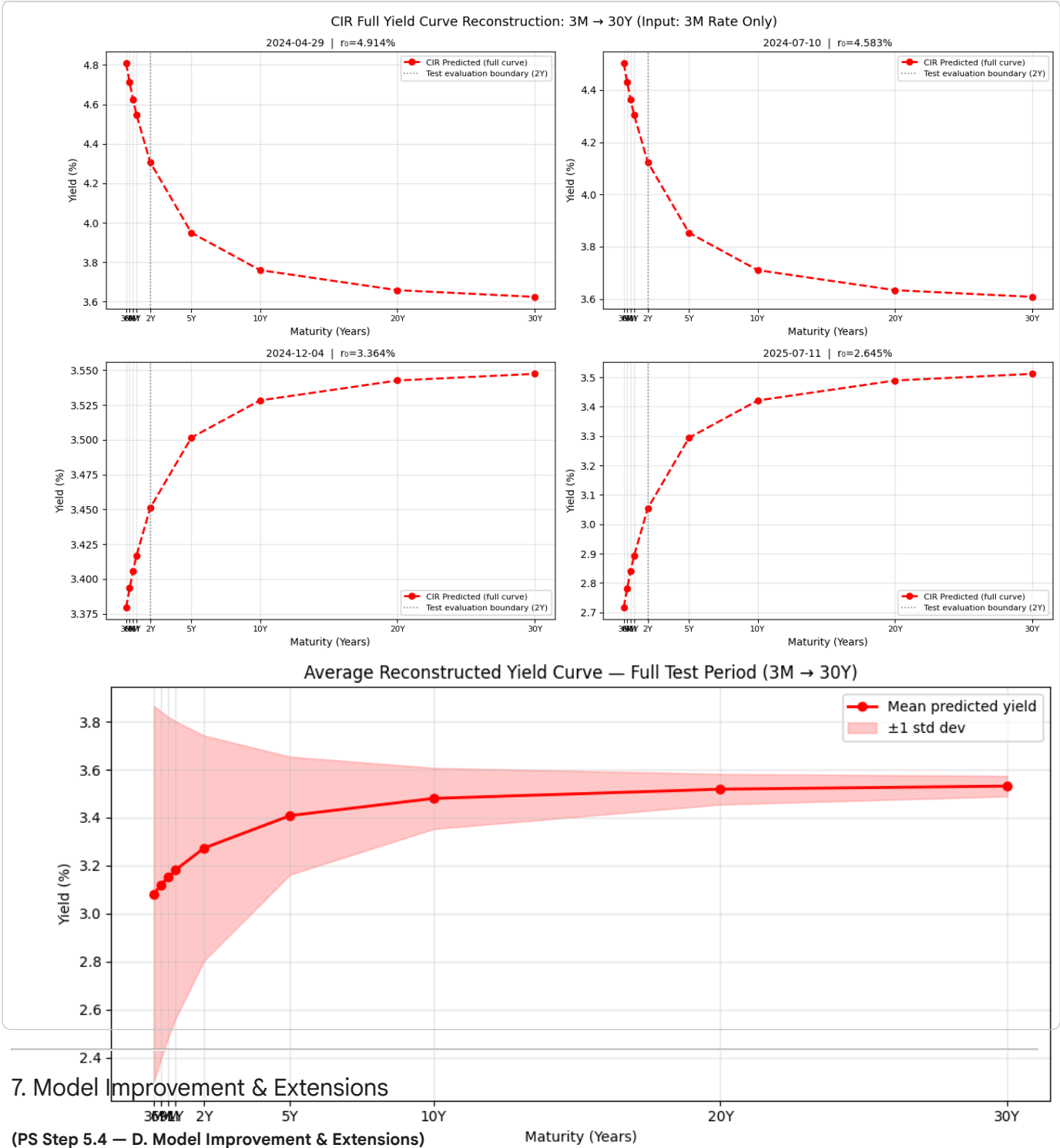
    axes[pi].plot(ALL_MATS, pred * 100, 'r--o', lw=2, label='CIR Predicted (full curve)')
    axes[pi].axvline(x=2.0, color='grey', ls=':', lw=1.2, label='Test evaluation boundary (2Y)')
    axes[pi].set_title(f'{date} | r0={r0*100:.3f}%', fontsize=10)
    axes[pi].set_xlabel('Maturity (Years)')
    axes[pi].set_ylabel('Yield (%)')
    axes[pi].set_xticks(ALL_MATS)
    axes[pi].set_xticklabels(MAT_LABELS, fontsize=8)
    axes[pi].legend(fontsize=8)
    axes[pi].grid(True, alpha=0.3)

fig.suptitle('CIR Full Yield Curve Reconstruction: 3M → 30Y (Input: 3M Rate Only)',
            fontsize=13)
plt.tight_layout()
plt.show()

# — Summary: average predicted curve level across test period —————
mean_pred = all_pred_full.mean(axis=0) * 100
std_pred = all_pred_full.std(axis=0) * 100

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(ALL_MATS, mean_pred, 'r-o', lw=2, label='Mean predicted yield')
ax.fill_between(ALL_MATS, mean_pred - std_pred, mean_pred + std_pred,
               alpha=0.2, color='red', label='±1 std dev')
ax.set_xlabel('Maturity (Years)')
ax.set_ylabel('Yield (%)')
ax.set_title('Average Reconstructed Yield Curve – Full Test Period (3M → 30Y)')
ax.set_xticks(ALL_MATS)
ax.set_xticklabels(MAT_LABELS)
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```



7. Model Improvement & Extensions

(PS Step 5.4 — D. Model Improvement & Extensions)

7.1 Limitations of the One-Factor CIR Model

Despite strong performance at short-to-medium maturities, the one-factor model has structural limitations:

- **Single stochastic driver:** All yield curve movements are parallel shifts driven by  $r_t$ . Real curves exhibit independent level, slope, and curvature changes.
- **Constrained curve shapes:** A single factor can only generate monotone or mildly humped curves — it cannot independently control short- and long-end dynamics.
- **Long-end underperformance:** 10Y–30Y yields incorporate inflation expectations, growth forecasts, term premia, and fiscal risk — none of which are recoverable from  $r_t$  alone.

7.2 Two-Factor CIR (Longstaff-Schwartz 1992)

We decompose the short rate into two independent CIR factors:

$$r_t = x_t + y_t$$

$$dx_t = \kappa_x(\theta_x - x_t)dt + \sigma_x\sqrt{x_t}dW_t^x$$

$$dy_t = \kappa_y(\theta_y - y_t)dt + \sigma_y\sqrt{y_t}dW_t^y$$

- **Factor  $x$  (level/policy):** High  $\kappa_x$  — captures short-term central bank policy effects that quickly revert to trend.
- **Factor  $y$  (slope/expectations):** Low  $\kappa_y$  — captures persistent long-run growth and inflation expectations.

Since  $W^x \perp W^y$ , bond prices factorise:  $P(t, T) = P_x(\tau, x) \cdot P_y(\tau, y)$ , giving yield:

$$y(\tau) = \frac{-(\ln P_x + \ln P_y)}{\tau}$$

```
def twofactor_yield(tau: float, x: float, y: float,
                    kappa_x: float, theta_x: float, sigma_x: float,
                    kappa_y: float, theta_y: float, sigma_y: float) -> float:
    """
    Two-Factor CIR yield for a single maturity  $\tau$ .
    Bond price factorises:  $P = P_x \cdot P_y$  (independent factors).
     $y(\tau) = -(\ln P_x + \ln P_y) / \tau$ 
    """
    t = np.array([tau])
    lnPx = np.log(max(cir_A(t, kappa_x, theta_x, sigma_x)[0], 1e-300)) - cir_B(t, kappa_x, sigma_x)[0] * x
    lnPy = np.log(max(cir_A(t, kappa_y, theta_y, sigma_y)[0], 1e-300)) - cir_B(t, kappa_y, sigma_y)[0] * y
    return -(lnPx + lnPy) / tau

def twofactor_curve(x: float, y: float,
                    kappa_x: float, theta_x: float, sigma_x: float,
                    kappa_y: float, theta_y: float, sigma_y: float,
                    maturities: list) -> np.ndarray:
    """Full yield curve prediction using two-factor CIR."""
    return np.array([twofactor_yield(tau, x, y,
                                     kappa_x, theta_x, sigma_x,
                                     kappa_y, theta_y, sigma_y)
                     for tau in maturities])

def calibrate_twofactor(r0: float, market_yields: np.ndarray,
                        maturities: list,
                        kappa_x_init: float, theta_x_init: float,
                        sigma_x_init: float) -> np.ndarray:
    """
    Calibrate all 8 two-factor parameters by minimising MSE against
    the observed term structure. Soft-constrains  $x + y \approx r0$ .
    """
    def objective(params):
        x, y, kx, tx, sx, ky, ty, sy = params
        if any(p <= 1e-8 for p in [x, y, kx, tx, sx, ky, ty, sy]):
            return 1e12
        penalty = 1e4 * (x + y - r0)**2
        pred = twofactor_curve(x, y, kx, tx, sx, ky, ty, sy, maturities)
        return np.sum((pred - market_yields)**2) + penalty

    x0 = [r0*0.65, r0*0.35,
          kappa_x_init, theta_x_init, sigma_x_init,
          kappa_y_init*0.4, theta_y_init*0.6, sigma_y_init*0.6]

    result = minimize(objective, x0=x0, method='Nelder-Mead',
                      options={'maxiter': 100000, 'xatol': 1e-9, 'fatol': 1e-9})
    return result.x

# Calibrate on last training day
print("Calibrating Two-Factor CIR...")
params_2f = calibrate_twofactor(r0_ref, market_ref, ALL_MATS,
                                kappa_cs, theta_cs, sigma_ols)
x_ref, y_ref, kx, tx, sx, ky, ty, sy = params_2f

print(f"\nFactor X (level) : x={x_ref:.4f}   $\kappa$ ={kx:.4f}   $\theta$ ={tx:.4f}   $\sigma$ ={sx:.4f}")
print(f"Factor Y (slope) : y={y_ref:.4f}   $\kappa$ ={ky:.4f}   $\theta$ ={ty:.4f}   $\sigma$ ={sy:.4f}")
print(f"x + y = {x_ref+y_ref:.4f}  r0 = {r0_ref:.4f}")
```

Calibrating Two-Factor CIR...

Factor X (level) : x=0.0308  $\kappa$ =0.4145  $\theta$ =0.0000  $\sigma$ =0.0326

```
Factor Y (slope) : y=0.0183  k=0.2356  0=0.0370  o=0.0076
x + y = 0.0492  r_o = 0.0492
```

### 7.3 Two-Factor Out-of-Sample Evaluation (up to 2Y)

```
def evaluate_twofactor(test_3m_df, test_actual_df,
                      x_ref, y_ref, r0_ref,
                      kx, tx, sx, ky, ty, sy,
                      r2_1f_per):
    """
    Evaluate Two-Factor CIR on the test set (maturities up to 2Y).
    Daily scaling: x_today = x_ref * (r0/r0_ref), same for y.
    """
    all_actual, all_predicted = [], []

    for i in range(len(test_3m_df)):
        r0 = test_3m_df.iloc[i]['ZC025YR']
        actual = test_actual_df.iloc[i][EVAL_COLS].values.astype(float)
        scale = r0 / r0_ref
        pred = twofactor_curve(x_ref*scale, y_ref*scale,
                               kx, tx, sx, ky, ty, sy, EVAL_MATS)

        all_actual.append(actual)
        all_predicted.append(pred)

    all_actual = np.array(all_actual)
    all_predicted = np.array(all_predicted)

    r2_per = []
    print("\n=== Two-Factor CIR - Out-of-Sample R² ===")
    print(f"{'Maturity':<10} {'2F R²':>8} {'1F R²':>8} {'Δ':>8}")
    print("-" * 38)
    for j, mat in enumerate(EVAL_MATS):
        r2_2 = r2_score(all_actual[:, j], all_predicted[:, j])
        r2_1 = r2_1f_per[j]
        r2_per.append(r2_2)
        print(f"{str(mat)+'Y':<10} {r2_2:>8.4f} {r2_1:>8.4f} {r2_2-r2_1:>+7.4f}")

    overall_2f = r2_score(all_actual.flatten(), all_predicted.flatten())
    overall_1f = r2_score(act_ols.flatten(), pred_ols.flatten())
    print(f"{'Overall':<10} {overall_2f:>8.4f} {overall_1f:>8.4f} {overall_2f-overall_1f:>+7.4f}")
    return all_actual, all_predicted, r2_per, overall_2f

act_2f, pred_2f, r2_2f_per, r2_2f = evaluate_twofactor(
    test_3m, test, x_ref, y_ref, r0_ref,
    kx, tx, sx, ky, ty, sy,
    r2_1f_per=r2_ols_per
)
```

```
=== Two-Factor CIR - Out-of-Sample R² ===
Maturity      2F R²      1F R²      Δ
-----
0.25Y         0.9976      0.9914  +0.0061
0.5Y          0.9960      0.9682  +0.0278
0.75Y         0.9811      0.9106  +0.0706
1.0Y          0.9462      0.8167  +0.1295
2.0Y          0.6334      0.0891  +0.5443
Overall       0.9536      0.8614  +0.0922
```

### 7.4 Two-Factor Full Yield Curve Prediction (All Maturities: 3M → 30Y)

```
# — Two-factor full curve predictions —————
all_pred_2f_full = []

for i in range(len(test_3m)):
    r0 = test_3m.iloc[i]['ZC025YR']
    scale = r0 / r0_ref
    pred = twofactor_curve(x_ref*scale, y_ref*scale,
                           kx, tx, sx, ky, ty, sy, ALL_MATS)
    all_pred_2f_full.append(pred)

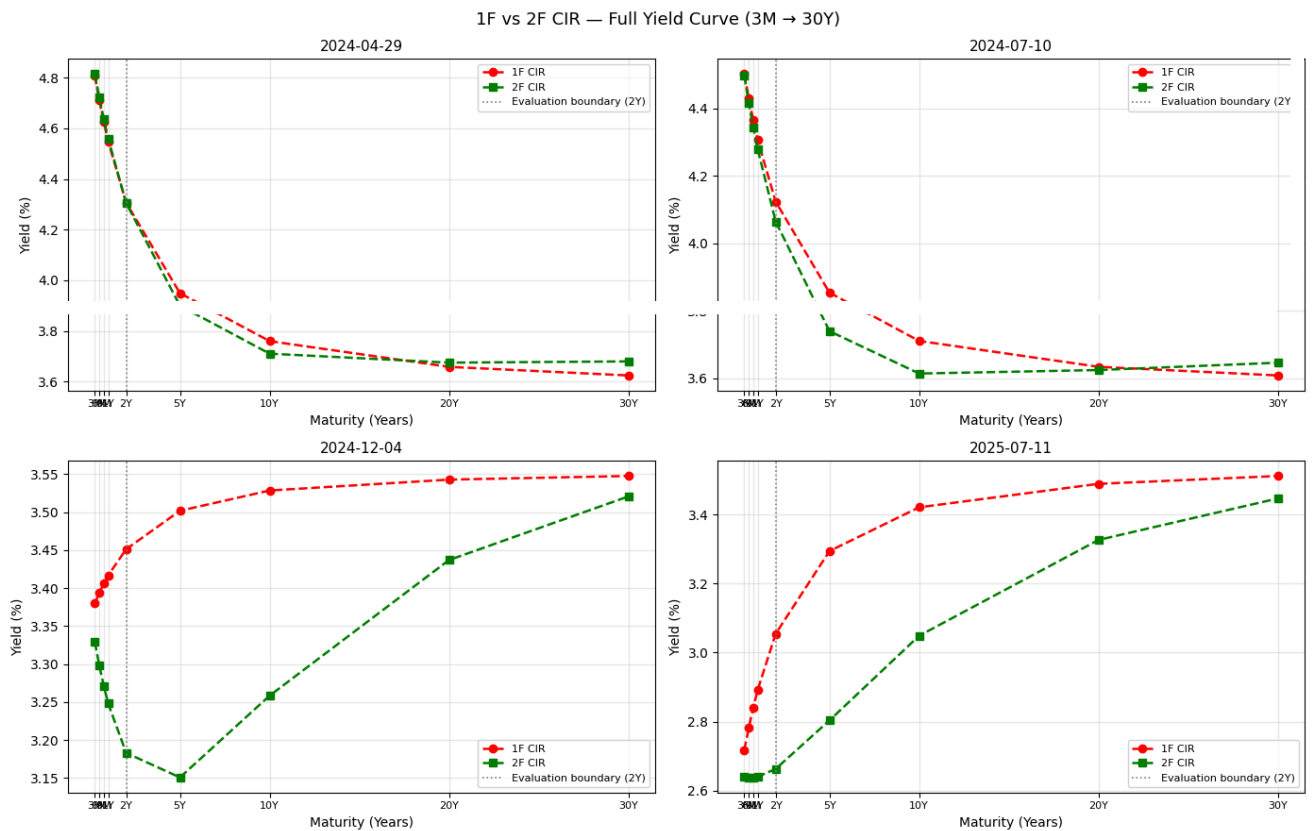
all_pred_2f_full = np.array(all_pred_2f_full)
```

```
# — Plot: 1F vs 2F full curve on sample dates —————
sample_idx = [0, 50, 150, 300]
fig, axes = plt.subplots(2, 2, figsize=(14, 9))
axes = axes.flatten()

for pi, di in enumerate(sample_idx):
    date = test_3m.iloc[di]['Date'].date()
    p1f = all_pred_full[di]
    p2f = all_pred_2f_full[di]

    axes[pi].plot(ALL_MATS, p1f*100, 'r--o', lw=1.8, label='1F CIR')
    axes[pi].plot(ALL_MATS, p2f*100, 'g--s', lw=1.8, label='2F CIR')
    axes[pi].axvline(x=2.0, color='grey', ls=':', lw=1.2, label='Evaluation boundary (2Y)')
    axes[pi].set_title(str(date), fontsize=11)
    axes[pi].set_xlabel('Maturity (Years)')
    axes[pi].set_ylabel('Yield (%)')
    axes[pi].set_xticks(ALL_MATS)
    axes[pi].set_xticklabels(MAT_LABELS, fontsize=8)
    axes[pi].legend(fontsize=8)
    axes[pi].grid(True, alpha=0.3)

fig.suptitle('1F vs 2F CIR — Full Yield Curve (3M → 30Y)', fontsize=13)
plt.tight_layout()
```



## 8. Results Summary & Critical Analysis

### (PS Step 5.5 — E. Critical Analysis)

#### 8.1 Results Comparison

```
# — Model comparison table —————
results = {
```